

0.1 Xử lý dữ liệu văn bản

Việc xử lý dữ liệu văn bản đóng vai trò nền tảng trong bất kỳ bài toán xử lý ngôn ngữ tự nhiên nào, bao gồm cả bài toán phân loại bình luận độc hại. Theo cuốn sách "Modern Information Retrieval" của Baeza-Yates và Ribeiro-Neto (1999), tiền xử lý văn bản là bước quan trọng để chuẩn hóa dữ liệu đầu vào và loại bỏ nhiễu, giúp cải thiện đáng kể chất lượng của các mô hình phân loại **baeza-yates1999modern**.

Quá trình tách từ bắt đầu bằng việc chuyển đổi văn bản gốc thành danh sách các token IDs, đồng thời tạo ra attention mask để đánh dấu các token thực sự so với các token đệm. Mỗi câu được thêm các token đặc biệt là [CLS] (classification) ở đầu và [SEP] (separator) ở cuối để phân tách các câu khi cần thiết. Độ dài tối đa của chuỗi được giới hạn ở 128 token, giúp cân bằng giữa khả năng nắm bắt ngữ cảnh và tốc độ huấn luyện. Attention mask sẽ có giá trị 1 cho các token thực và 0 cho các token đệm, cho phép mô hình bỏ qua phần đệm khi tính toán.

Trong bài toán phân loại đa nhãn, mỗi bình luận được gán nhãn bởi 6 cột nhị phân tương ứng với 6 loại độc hại: toxic, severe_toxic, obscene, threat, insult, và identity_hate. Điều đáng chú ý là một bình luận có thể đồng thời thuộc nhiều nhãn. Điều này phân biệt bài toán phân loại đa nhãn với phân loại đa lớp, đòi hỏi việc sử dụng Binary Cross-Entropy loss thay vì Categorical Cross-Entropy **goodfellow2016deep**. Hàm mất mát BCEWithLogitsLoss được tính như sau:

$$\mathcal{L}_{BCE} = -\frac{1}{N \times C} \sum_{i=1}^N \sum_{c=1}^C [y_{ic} \log(\sigma(x_{ic})) + (1 - y_{ic}) \log(1 - \sigma(x_{ic}))] \quad (1)$$

Trong đó N là số mẫu, C là số lớp (6), y_{ic} là nhãn thực của mẫu i lớp c , x_{ic} là logits đầu ra của mô hình, và σ là hàm sigmoid. Khi sử dụng trọng số lớp (pos_weight), công thức trở thành:

$$\mathcal{L}_{BCE}^{weighted} = -\frac{1}{N \times C} \sum_{i=1}^N \sum_{c=1}^C w_c [y_{ic} \log(\sigma(x_{ic})) + (1 - y_{ic}) \log(1 - \sigma(x_{ic}))] \quad (2)$$

Với w_c là trọng số của lớp c , được tính bằng tỷ lệ $\frac{\text{số mẫu âm}_c}{\text{số mẫu dương}_c}$.

0.2 Các mô hình học sâu

0.2.1 Long Short-Term Memory (LSTM)

Mạng Long Short-Term Memory (LSTM) là một biến thể của mạng RNN (Recurrent Neural Network) được giới thiệu bởi Hochreiter và Schmidhuber vào năm 1997, nhằm giải quyết hai vấn đề cơ bản của RNN truyền thống: tiêu biến gradient và bùng nổ gradient **hochreiter1997lstm**. Khi chuỗi đầu vào quá dài, gradient trong quá trình lan truyền ngược qua thời gian (BPTT) có xu hướng giảm rất nhanh hoặc tăng rất nhanh, khiến mô hình không thể học được các phụ thuộc từ xa trong chuỗi.

Kiến trúc cốt lõi của LSTM dựa trên khái niệm cell state (trạng thái ô), đóng vai trò như một "băng truyền" cho phép thông tin chảy dọc theo chuỗi mà hầu như không bị thay đổi. Cell state được điều khiển bởi ba cổng: Forget Gate, Input Gate, và Output Gate.

Các công thức toán học của LSTM được biểu diễn như sau:

$$f_t = \sigma(W_{hf}h_{t-1} + W_{xf}x_t + b_f) \quad (\text{Forget Gate}) \quad (3)$$

Forget Gate quyết định thông tin nào từ cell state trước đó cần được loại bỏ. Hàm sigmoid σ đưa ra giá trị trong khoảng $[0,1]$, trong đó 0 có nghĩa là "loại bỏ hoàn toàn" và 1 có nghĩa là "giữ lại hoàn toàn".

$$i_t = \sigma(W_{hi}h_{t-1} + W_{xi}x_t + b_i) \quad (\text{Input Gate}) \quad (4)$$

Input Gate quyết định thông tin mới nào sẽ được thêm vào cell state.

$$\tilde{c}_t = \tanh(W_{hc}h_{t-1} + W_{xc}x_t + b_c) \quad (\text{Candidate Cell State}) \quad (5)$$

Candidate Cell State tạo ra các giá trị ứng viên để thêm vào cell state. Hàm tanh đưa ra giá trị trong khoảng $[-1,1]$, cho phép cả giá trị dương và âm.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Cell State Update}) \quad (6)$$

Cell State mới được cập nhật bằng cách kết hợp thông tin cũ (được điều khiển bởi Forget Gate) và thông tin mới (được điều khiển bởi Input Gate). Toán tử $\odot$ là phép nhân Hadamard (theo từng phần tử).

$$o_t = \sigma(W_{ho}h_{t-1} + W_{xo}x_t + b_o) \quad (\text{Output Gate}) \quad (7)$$

Output Gate quyết định phần nào của cell state sẽ được xuất ra.

$$h_t = o_t \odot \tanh(c_t) \quad (\text{Hidden State}) \quad (8)$$

Hidden State mới là kết quả của việc nhân Output Gate với tanh của Cell State, quyết định thông tin nào được truyền ra ngoài.

0.2.2 Bidirectional LSTM (BiLSTM)

Bidirectional LSTM (BiLSTM) là một mở rộng của LSTM, cho phép xử lý chuỗi đầu vào theo cả hai hướng cùng lúc: từ trái sang phải (tiền) và từ phải sang trái (lùi) **graves2012supervised**. Ý tưởng cốt lõi là mỗi từ trong câu không chỉ phụ thuộc vào ngữ cảnh đứng trước nó mà còn phụ thuộc vào ngữ cảnh đứng sau nó. Ví dụ trong câu "I don't hate this movie", từ "hate" mang nghĩa tiêu cực, nhưng trong ngữ cảnh đầy đủ "don't hate" lại mang nghĩa tích cực.

Về mặt kỹ thuật, BiLSTM bao gồm hai mạng LSTM chạy song song. Công thức tính hidden state của BiLSTM tại mỗi vị trí t :

$$\vec{h}_t = \text{LSTM}_{\rightarrow}(x_t, \vec{h}_{t-1}) \quad (\text{Forward}) \quad (9)$$

$$\overleftarrow{h}_t = \text{LSTM}_{\leftarrow}(x_t, \overleftarrow{h}_{t+1}) \quad (\text{Backward}) \quad (10)$$

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad (\text{Concatenation}) \quad (11)$$

Hidden state của BiLSTM được tạo bằng cách nối hidden state từ LSTM hướng tiến và hidden state từ LSTM hướng lùi tại vị trí đó. Do đó, kích thước hidden state của BiLSTM gấp đôi so với LSTM thông thường: $\text{hidden_dim}_{BiLSTM} = 2 \times \text{hidden_dim}_{LSTM}$.

Pooling cuối cùng sử dụng Global Average Pooling để tổng hợp thông tin từ tất cả các bước thời gian. Với BiLSTM, mỗi hidden state đã có chiều gấp đôi, nên vector đầu ra sau pooling có kích thước $\text{hidden_size} \times 2$.

0.2.3 Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) là một biến thể khác của RNN, được đề xuất bởi Cho vào năm 2014 trong bài báo về RNN Encoder-Decoder cho dịch máy thống kê **cho2014gru**, với thiết kế đơn giản hóa hơn so với LSTM. GRU sử dụng ít cổng hơn (chỉ 2 cổng thay vì 3) và không sử dụng cell state riêng biệt, chỉ dùng hidden state duy nhất, giúp giảm số lượng tham số cần học khoảng 25%.

Các công thức toán học của GRU:

$$z_t = \sigma(W_{hz}h_{t-1} + W_{xz}x_t + b_z) \quad (\text{Update Gate}) \quad (12)$$

Update Gate z_t kết hợp vai trò của Forget Gate và Input Gate trong LSTM. Khi z_t gần 1, mô hình giữ lại hầu hết thông tin cũ; khi z_t gần 0, mô hình thêm nhiều thông tin mới.

$$r_t = \sigma(W_{hr}h_{t-1} + W_{xr}x_t + b_r) \quad (\text{Reset Gate}) \quad (13)$$

Reset Gate r_t kiểm soát mức độ thông tin quá khứ cần được bỏ qua khi tính toán hidden state ứng viên mới. Nếu r_t gần 0, mô hình bỏ qua hoàn toàn hidden state trước đó.

$$\tilde{h}_t = \tanh(W_{hn}(r_t \odot h_{t-1}) + W_{xn}x_t + b_n) \quad (\text{Candidate Hidden State}) \quad (14)$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1} \quad (\text{Hidden State Update}) \quad (15)$$

Hidden state mới là sự nội suy giữa trạng thái ứng viên và hidden state trước đó, được điều khiển bởi update gate. Công thức này cho phép mô hình linh hoạt quyết định bao nhiêu thông tin cũ cần được giữ lại.

Ưu điểm chính của GRU so với LSTM là ít tham số hơn, dẫn đến huấn luyện nhanh hơn và ít nguy cơ quá khớp hơn. Trong nhiều thử nghiệm thực tế, hiệu suất của GRU và LSTM thường tương đương nhau, và sự lựa chọn giữa hai kiến trúc phụ thuộc vào đặc điểm cụ thể của bài toán và dữ liệu.

0.2.4 Cơ chế Attention

Cơ chế Attention (chú ý) là một trong những đột phá quan trọng nhất trong lĩnh vực học sâu, được giới thiệu ban đầu trong bài toán dịch máy nơ-ron bởi Bahdanau

và cộng sự vào năm 2014 **bahdanau2014attention**. Ý tưởng cốt lõi của Attention là thay vì dựa hoàn toàn vào hidden state cuối cùng để đại diện cho toàn bộ chuỗi (như trong RNN truyền thống), mô hình có thể "chú ý" (tập trung) vào các phần khác nhau của chuỗi đầu vào với mức độ khác nhau.

Trong bài toán phân loại bình luận độc hại, không phải từ nào cũng quan trọng như nhau. Ví dụ trong câu "You are an idiot", từ "idiot" mang ý nghĩa quan trọng nhất để xác định bình luận là toxic, trong khi các từ như "are", "an" gần như không cung cấp thông tin hữu ích.

Công thức toán học của Self-Attention được sử dụng trong đề tài:

$$e_t = \tanh(W_a h_t + b_a) \quad (\text{Energy Score}) \quad (16)$$

Bước đầu tiên tính các điểm năng lượng e_t cho mỗi bước thời gian, trong đó h_t là hidden state tại bước thời gian t và W_a là ma trận trọng số có thể huấn luyện. Điểm năng lượng đại diện cho mức độ "quan trọng" của mỗi bước thời gian.

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^T \exp(e_k)} = \text{softmax}(e_t) \quad (\text{Attention Weights}) \quad (17)$$

Attention weights α_t được tính bằng softmax của các điểm năng lượng, đảm bảo tổng các trọng số bằng 1 ($\sum_{t=1}^T \alpha_t = 1$). Điều này cho phép tính giải thích được: ta có thể biết được mô hình đang tập trung vào phần nào của văn bản.

$$c = \sum_{t=1}^T \alpha_t h_t \quad (\text{Context Vector}) \quad (18)$$

Context vector c là tổng có trọng số của tất cả hidden states. Context vector này được sử dụng thay cho mean pooling hoặc hidden state cuối cùng làm đầu vào cho lớp phân loại. Điểm khác biệt chính so với mean pooling ($\bar{h} = \frac{1}{T} \sum h_t$) là attention cho phép mô hình học để tập trung vào các phần quan trọng.

Yang và cộng sự (2016) đã phát triển kiến trúc Hierarchical Attention Networks sử dụng attention ở nhiều cấp độ và đạt kết quả ấn tượng trên bài toán phân loại văn bản **yang2016hierarchical**.

0.2.5 Attention BiLSTM

Attention BiLSTM là sự kết hợp giữa kiến trúc BiLSTM và cơ chế Attention, tận dụng ưu điểm của cả hai. BiLSTM cung cấp khả năng nắm bắt ngữ cảnh hai chiều, trong khi Attention cho phép mô hình tập trung vào các từ quan trọng nhất

khi đưa ra quyết định phân loại.

Trong mô hình này, đầu ra của BiLSTM là một chuỗi các hidden states (mỗi hidden state có chiều gấp đôi do nối hai hướng), sau đó Self-Attention được áp dụng lên chuỗi này để tạo ra một context vector duy nhất đại diện cho toàn bộ câu. Attention weights được tính dựa trên các hidden states từ BiLSTM, cho phép mô hình học được tổ hợp ngữ cảnh hai chiều của từng từ trước khi quyết định mức độ quan trọng.

0.2.6 Recurrent Convolutional Neural Network (RCNN)

RCNN (Recurrent Convolutional Neural Network) là kiến trúc kết hợp giữa RNN và CNN, được đề xuất bởi Lai và cộng sự vào năm 2015 cho bài toán phân loại văn bản tại hội nghị AAAI **lai2015rcnn**. Ý tưởng cốt lõi của RCNN là sử dụng BiLSTM để thu thập ngữ cảnh hai chiều cho mỗi từ, sau đó kết hợp ngữ cảnh đó với embedding gốc của từ để tạo ra biểu diễn phong phú hơn, rồi cuối cùng sử dụng Max Pooling thay vì Average Pooling để trích xuất các đặc trưng quan trọng nhất.

Kiến trúc RCNN trong đề tài bao gồm các bước sau:

Bước 1: BiLSTM tạo ngữ cảnh hai chiều

$$c_L(t) = \overrightarrow{h_t} \quad (\text{Ngữ cảnh trái tại vị trí } t) \quad (19)$$

$$c_R(t) = \overleftarrow{h_t} \quad (\text{Ngữ cảnh phải tại vị trí } t) \quad (20)$$

Bước 2: Nối ngữ cảnh với embedding

$$x_t^{combined} = [c_L(t); e_t; c_R(t)] \quad (\text{Biểu diễn kết hợp}) \quad (21)$$

Vector biểu diễn tại mỗi vị trí được tạo bằng cách nối ngữ cảnh trái, embedding gốc, và ngữ cảnh phải. Kích thước của vector này là: $\dim(c_L) + \dim(e_t) + \dim(c_R) = 2 \times \text{hidden_size} + \text{embedding_dim}$.

Bước 3: Tầng Fusion với tanh

$$y_t = \tanh(W_f \cdot x_t^{combined} + b_f) \quad (\text{Fusion}) \quad (22)$$

Tầng Fusion tích hợp các đặc trưng từ ba nguồn (ngữ cảnh trái, embedding, ngữ cảnh phải) thành một vector biểu diễn duy nhất cho mỗi vị trí.

Bước 4: Max Pooling

$$\hat{y} = \max_{t \in [1, T]} y_t \quad (\text{Global Max Pooling}) \quad (23)$$

Global Max Pooling lấy giá trị lớn nhất theo chiều thời gian, giúp tập trung vào những đặc trưng nổi bật nhất.

0.2.7 Transformer Encoder

Transformer là kiến trúc được giới thiệu trong bài báo "Attention Is All You Need" của Vaswani và cộng sự vào năm 2017 tại hội nghị NeurIPS, đánh dấu một bước tiến lớn trong lĩnh vực xử lý ngôn ngữ tự nhiên **vaswani2017transformer**. Điểm khác biệt cốt lõi của Transformer so với các mô hình RNN là nó không sử dụng cơ chế tuần tự, mà thay vào đó xử lý toàn bộ chuỗi đầu vào song song trong một bước duy nhất.

Do Transformer không xử lý tuần tự nên nó không có thông tin về vị trí của các từ trong câu một cách tự nhiên. Positional Encoding được sử dụng để bổ sung thông tin vị trí:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \quad (\text{Vị trí chẵn}) \quad (24)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \quad (\text{Vị trí lẻ}) \quad (25)$$

Trong đó pos là vị trí của từ, i là chiều của embedding, và d là tổng chiều của embedding. Các hàm sine và cosine với các tần số khác nhau tạo ra một vector duy nhất cho mỗi vị trí mà mô hình có thể học được.

Self-Attention là cơ chế cốt lõi của Transformer. Công thức attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (26)$$

Trong đó Q (Query), K (Key), và V (Value) là ba ma trận được tính từ đầu vào: $Q = XW_Q$, $K = XW_K$, $V = XW_V$. Hệ số $\sqrt{d_k}$ trong mẫu số giúp ổn định gradient trong quá trình huấn luyện khi d_k lớn.

Multi-Head Attention chạy nhiều đầu attention song song:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (27)$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i) \quad (28)$$

Mỗi head với một không gian con khác nhau, cho phép mô hình học các loại quan hệ khác nhau đồng thời: head này có thể học quan hệ ngữ pháp, head kia học quan hệ ngữ nghĩa, và một head khác học các mẫu phủ định.

Transformer Encoder Block bao gồm hai thành phần chính với Residual Connection và Layer Normalization:

$$x' = \text{LayerNorm}(x + \text{MultiHead}(x, x, x)) \quad (29)$$

$$x'' = \text{LayerNorm}(x' + \text{FFN}(x')) \quad (30)$$

Trong đó Feed-Forward Network (FFN) gồm hai lớp tuyến tính với hàm kích hoạt ReLU:

$$\text{FFN}(x) = W_2 \cdot \text{ReLU}(W_1 \cdot x + b_1) + b_2 \quad (31)$$

Layer Normalization **ba2016layer** được tính như sau:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \quad (32)$$

Trong đó $\mu = \frac{1}{d} \sum_{i=1}^d x_i$ và $\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$ lần lượt là trung bình và phương sai của đầu vào. Các tham số γ và β là các tham số có thể học được.

Pre-Layer Normalization (Pre-LN): Khác với Post-LN được mô tả ở trên (LayerNorm sau residual), các phiên bản cải tiến của Transformer sử dụng Pre-LN, áp dụng LayerNorm trước sublayer:

$$x' = x + \text{MultiHead}(\text{LayerNorm}(x), \text{LayerNorm}(x), \text{LayerNorm}(x)) \quad (33)$$

$$x'' = x' + \text{FFN}(\text{LayerNorm}(x')) \quad (34)$$

Xiong và cộng sự (2020) **xiong2020layer** đã chứng minh Pre-LN vượt trội so với Post-LN, đặc biệt với các Transformer nhiều tầng. Pre-LN giúp gradient flow ổn định hơn trong quá trình lan truyền ngược, tránh bùng nổ gradient ở các tầng gần đầu vào.

CLS Token: Thay vì sử dụng mean pooling trên tất cả các đầu ra, phiên bản cải tiến thêm một token đặc biệt [CLS] vào đầu chuỗi. Token này có khả năng học để tổng hợp thông tin toàn bộ câu thông qua cơ chế self-attention, và đầu ra tại vị trí này được sử dụng trực tiếp cho phân loại. Phương pháp này được sử dụng phổ biến

trong BERT **devlin2019bert** và các mô hình dựa trên Transformer khác.

Trong đề tài này, Transformer được cấu hình với Pre-LN, 4 tầng mã hóa, CLS token, và đầu ra tại vị trí CLS được dùng làm biểu diễn cho phân loại.

0.3 Các phương pháp tối ưu hóa huấn luyện

0.3.1 Xử lý dữ liệu mất cân bằng với Class-Weighted Loss

Một trong những thách thức phổ biến trong các bài toán phân loại là sự mất cân bằng giữa các lớp. Khi số lượng mẫu của các lớp chênh lệch lớn, mô hình có xu hướng thiên về lớp đa số và bỏ qua lớp thiểu số.

Để giải quyết vấn đề này, đề tài sử dụng Binary Cross-Entropy with Logits Loss (BCEWithLogitsLoss) với trọng số lớp (pos_weight). Trọng số cho mỗi lớp được tính bằng:

$$w_c = \frac{\text{số mẫu âm}_c}{\text{số mẫu dương}_c} = \frac{N - p_c}{p_c} \quad (35)$$

Trong đó p_c là số mẫu dương tính của lớp c và N là tổng số mẫu. Trọng số này nhân với loss của các mẫu dương, khiến việc bỏ sót một mẫu độc hại có chi phí cao hơn nhiều so với việc dự đoán sai một mẫu không độc hại.

Để tránh trọng số quá lớn gây ra việc phạt quá mức lớp đa số, đề tài áp dụng clipping:

$$w_c^{clipped} = \min(w_c, \text{max_weight}) \quad (36)$$

Lin và cộng sự (2017) đã nghiên cứu về Focal Loss như một phương pháp hiệu quả để xử lý dữ liệu mất cân bằng **lin2017focal**:

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (37)$$

Trong đó p_t là xác suất của lớp đúng, α_t là trọng số cân bằng, và γ là tham số tập trung. Focal Loss giảm trọng số của các mẫu dễ phân loại và tập trung vào các mẫu khó.

Sokolova và Lapalme (2009) đã phân tích chi tiết các độ đo hiệu suất cho bài toán phân loại và nhận thấy rằng F1-score đặc biệt hữu ích khi dữ liệu mất cân bằng **sokolova2009systematic**:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (38)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (39)$$

F1 Macro là trung bình F1 của các lớp: $F1_{macro} = \frac{1}{C} \sum_{c=1}^C F1_c$.

0.3.2 Masked Pooling

Pooling là bước quan trọng để tổng hợp thông tin từ các bước thời gian của chuỗi thành một vector đại diện duy nhất. Tuy nhiên, khi sử dụng đệm để đồng nhất độ dài chuỗi trong batch, các token đệm (giá trị 0) sẽ làm "pha loãng" biểu diễn cuối cùng nếu pooling được tính trên toàn bộ độ dài.

Trong các phiên bản đầu, cả mean pooling và max pooling đều tính trên toàn bộ chuỗi, bao gồm cả các token đệm. Điều này dẫn đến việc các chuỗi ngắn có đến 50% token là đệm, làm giảm đáng kể chất lượng biểu diễn. Để khắc phục, đề tài sử dụng masked pooling, tận dụng attention_mask (mảng nhị phân với 1 cho token thực và 0 cho token đệm):

Masked Mean Pooling:

$$\bar{h} = \frac{\sum_{t=1}^T m_t \cdot h_t}{\sum_{t=1}^T m_t + \epsilon} \quad (40)$$

Masked Max Pooling:

$$\tilde{h}_i = \max_{t:m_t=1} h_{t,i} \quad (41)$$

Trong đó m_t là giá trị attention mask tại bước thời gian t , h_t là hidden state của RNN. Với max pooling, các token đệm được gán $-\infty$ trước khi lấy max, đảm bảo chúng không ảnh hưởng đến kết quả.

0.3.3 Focal Loss

Focal Loss **lin2017focal** là một cải tiến của Binary Cross-Entropy, được thiết kế để xử lý dữ liệu mất cân bằng bằng cách giảm trọng số của các mẫu dễ phân loại:

$$\mathcal{L}_{focal} = -\frac{1}{N} \sum_{i=1}^N (1 - p_{t,i})^\gamma \log(p_{t,i}) \quad (42)$$

Với $\gamma = 2$, một mẫu có $p_t = 0.9$ (dễ) sẽ có trọng số $(1 - 0.9)^2 = 0.01$, trong khi mẫu có $p_t = 0.1$ (khó) có trọng số $(1 - 0.1)^2 = 0.81$. Điều này giúp mô hình tập trung vào các mẫu khó (thường là các lớp thiểu số) và cải thiện F1 Macro trên các lớp này.

0.3.4 Label Smoothing

Label Smoothing là kỹ thuật regularization được đề xuất bởi Szegedy và cộng sự (2016) **szegedy2016rethinking**, nhằm giảm quá khớp và cải thiện hiệu chỉnh của mô hình. Thay vì sử dụng nhãn one-hot cứng ($y_c \in \{0, 1\}$), label smoothing tạo ra nhãn mềm:

$$y_c^{smooth} = y_c \cdot (1 - \epsilon) + \frac{\epsilon}{C} \quad (43)$$

Với $\epsilon = 0.1$, nhãn 1 trở thành $1 - \epsilon + \epsilon/C$ và nhãn 0 trở thành ϵ/C . Điều này ngăn mô hình trở nên quá tự tin và cải thiện khả năng tổng quát hóa.

0.3.5 Gradient Accumulation

Gradient Accumulation cho phép mô phỏng kích thước batch hiệu quả lớn hơn khi bộ nhớ GPU hạn chế. Thay vì cập nhật tham số sau mỗi batch nhỏ (micro-batch), gradient được tích lũy qua k batch trước khi thực hiện một bước tối ưu:

$$g_{accum} = \frac{1}{k} \sum_{i=1}^k \nabla_{\theta} \mathcal{L}_i \quad (44)$$

$$\theta \leftarrow \theta - \eta \cdot \text{AdamW}(\theta, g_{accum}) \quad (45)$$

Với $k = 4$ và micro-batch size = 64, kích thước batch hiệu quả đạt 256, mang lại gradient ổn định hơn và cải thiện chất lượng hội tụ. Smith và cộng sự (2018) **smith2018superconv** đã chứng minh rằng kích thước batch hiệu quả lớn giúp huấn luyện ổn định và có thể đạt được hiệu suất tương đương với kích thước batch thực tế.

0.3.6 AdamW Optimizer với Weight Decay

Adam (Adaptive Moment Estimation) được giới thiệu bởi Kingma và Ba năm 2015 **kingma2015adam**, kết hợp Momentum và RMSprop:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{First moment estimate}) \quad (46)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{Second moment estimate}) \quad (47)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{Bias correction}) \quad (48)$$

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (\text{Parameter update}) \quad (49)$$

AdamW (Adam with Decoupled Weight Decay) được đề xuất bởi Loshchilov và Hutter năm 2017 **loshchilov2017adamw**, tách biệt weight decay khỏi gradient

update:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \cdot \lambda \cdot \theta_t \quad (50)$$

Trong đó λ là hệ số weight decay (10^{-4} trong đề tài). Việc tách biệt weight decay giúp đảm bảo regularization effect nhất quán bất kể giá trị của các thành phần thích nghi.

Weight decay (L2 regularization) đóng vai trò quan trọng trong việc ngăn chặn quá khớp. Nó khuyến khích các weights có giá trị nhỏ, giúp cải thiện khả năng tổng quát hóa của mô hình **goodfellow2016deep**.

Srivastava và cộng sự (2014) đã nghiên cứu về Dropout như một phương pháp regularization hiệu quả khác **srivastava2014dropout**. Trong đề tài, dropout với tỷ lệ 0.3 được áp dụng trước lớp fully connected cuối cùng để ngăn quá khớp.

0.3.7 Gradient Clipping

Gradient Clipping là kỹ thuật quan trọng đặc biệt đối với các mô hình RNN. Trong quá trình lan truyền ngược qua thời gian, gradient được nhân qua nhiều bước thời gian liên tiếp, có thể dẫn đến bùng nổ gradient.

Kỹ thuật Gradient Clipping giới hạn giá trị của gradient bằng cách điều chỉnh tỷ lệ nếu norm vượt quá ngưỡng:

$$\text{if } \|g\|_2 > \text{max_norm} : \quad g \leftarrow g \cdot \frac{\text{max_norm}}{\|g\|_2} \quad (51)$$

Trong đó $\|g\|_2 = \sqrt{\sum_i g_i^2}$ là L2 norm của gradient. Trong đề tài, max_norm = 1.0 được sử dụng. Gradient clipping được áp dụng sau khi gọi loss.backward() và trước khi gọi optimizer.step().

Graves và cộng sự (2012) đã áp dụng gradient clipping thành công trong các mạng RNN cho nhận dạng giọng nói và nhận thấy rằng kỹ thuật này đặc biệt quan trọng khi huấn luyện trên các chuỗi dài **graves2012supervised**.

LeCun và cộng sự (1998) đã thảo luận chi tiết về các thủ thuật huấn luyện mạng nơ-ron, bao gồm tầm quan trọng của việc kiểm soát gradient để đảm bảo quá trình huấn luyện ổn định **lecun1998gradient**.

0.3.8 OneCycleLR

OneCycleLR được đề xuất bởi Smith và Topin (2018) **smith2018superconvergence**, là một chiến lược điều chỉnh tốc độ học dựa trên nguyên lý "hội tụ siêu tốc". Thay

vì warmup rồi giảm dần, OneCycleLR thực hiện đúng một chu kỳ: tăng tốc độ học từ lr_{min} lên lr_{max} trong giai đoạn đầu, sau đó giảm về lr_{min} trong giai đoạn cuối.

$$lr(t) = \begin{cases} lr_{min} + (lr_{max} - lr_{min}) \cdot \frac{t}{T_{warmup}}, & 0 \leq t < T_{warmup} \\ lr_{max} - (lr_{max} - lr_{min}) \cdot \left[1 - \frac{1}{2} \left(1 + \cos \left(\pi \cdot \frac{t - T_{warmup}}{T_{total} - T_{warmup}} \right) \right) \right], & T_{warmup} \leq t < T_{total} \end{cases} \quad (52)$$

Trong cấu hình đề tài: $lr_{max} = 1 \times 10^{-3}$, $pct_{start} = 0.1$ (warmup chiếm 10% tổng số bước), và T_{total} là tổng số bước batch. Khác với Warmup Cosine Scheduler giảm dần đến 0, OneCycleLR chỉ có đúng một chu kỳ tăng-giảm duy nhất trong toàn bộ quá trình huấn luyện.

Smith (2018) **smith2018superconvergence** đã chứng minh rằng OneCycleLR giúp mô hình hội tụ nhanh hơn đáng kể so với các bộ lập lịch truyền thống, có thể đạt được độ chính xác tương đương chỉ với một vài epoch. Nguyên lý hoạt động dựa trên việc cho phép mô hình "thoát khỏi" các cực tiểu địa phương hẹp ở giai đoạn đầu với tốc độ học cao, sau đó "hạ cánh" vào một cực tiểu địa phương rộng và phẳng hơn ở giai đoạn cuối với tốc độ học thấp.

0.3.9 Early Stopping

Early Stopping là kỹ thuật regularization đơn giản nhưng hiệu quả, nhằm ngăn chặn quá khớp bằng cách dừng quá trình huấn luyện khi độ đo kiểm định không còn cải thiện.

Đề tài sử dụng Early Stopping với cấu hình:

- **patience** = 2 epoch
- **mode** = 'max' (theo dõi ROC-AUC)
- **Độ đo** = ROC-AUC (thay vì loss)

ROC-AUC (Receiver Operating Characteristic - Area Under Curve) đo lường khả năng phân biệt của mô hình giữa các lớp dương tính và âm tính:

$$AUC = \int_0^1 TPR(t) dFPR(t) \quad (53)$$

Trong đó TPR (True Positive Rate) = Recall và FPR (False Positive Rate) = $\frac{FP}{FP+TN}$. AUC bằng 0.5 có nghĩa là dự đoán ngẫu nhiên, và AUC bằng 1.0 có nghĩa

là phân loại hoàn hảo **fawcett2006introduction**.

Lý do sử dụng ROC-AUC thay vì loss là vì loss có thể bị ảnh hưởng bởi trọng số lớp, trong khi ROC-AUC đo lường khả năng phân biệt thực sự của mô hình một cách độc lập với ngưỡng. Raschka (2018) đã thảo luận chi tiết về tầm quan trọng của việc sử dụng các độ đo phù hợp để tránh quá khớp **raschka2018roc**.

Input: *patience* *p*, *best_score*, *current_score*

if *current_score* > *best_score* **then**

best_score ← *current_score*

patience_counter ← 0

 Lưu model checkpoint

else

patience_counter ← *patience_counter* + 1

if *patience_counter* ≥ *p* **then**

 Khôi phục model từ checkpoint tốt nhất

 Dừng huấn luyện

Algorithm 1: Thuật toán Early Stopping

Wolpert (1997) đã chứng minh No Free Lunch Theorems, cho thấy rằng không có thuật toán tối ưu nhất cho mọi bài toán **wolpert1997no**. Early Stopping là một ví dụ về việc áp dụng nguyên tắc này trong thực tế.

0.3.10 Tìm kiếm ngưỡng tối ưu

Trong phân loại đa nhãn với đầu ra là xác suất từ hàm sigmoid, ngưỡng mặc định để chuyển đổi xác suất thành nhãn nhị phân thường là 0.5. Tuy nhiên, với dữ liệu mất cân bằng, ngưỡng 0.5 không phải lúc nào cũng tối ưu.

Đề tài áp dụng kỹ thuật tìm kiếm ngưỡng tối ưu riêng biệt cho từng lớp:

$$\theta_c^* = \arg \max_{\theta \in [0.1, 0.9]} F1_c(\theta) \quad (54)$$

Với mỗi lớp *c*, duyệt qua các giá trị ngưỡng từ 0.1 đến 0.9 với bước nhảy 0.01 và chọn ngưỡng tối ưu dựa trên F1-score cao nhất. Phương pháp này cho phép mỗi lớp có một ngưỡng riêng phù hợp với phân phối và đặc điểm của lớp đó.

Fawcett (2006) đã thảo luận chi tiết về tầm quan trọng của việc lựa chọn ngưỡng phù hợp trong các bài toán phân loại và cách nó ảnh hưởng đến đường cong ROC và các độ đo hiệu suất khác nhau **fawcett2006introduction**.